

INCEPTION

Complete Step-by-Step Build Guide

42 School Project — Docker Infrastructure

A professional DevOps guide to build, understand, and defend the Inception project from scratch.

■ Docker & Docker Compose	Understand images, containers, networking, volumes
■ TLS / NGINX	Serve HTTPS with self-signed certificates
■ MariaDB	Database container, users, persistence
■ WordPress + PHP-FPM	CMS wired to NGINX and MariaDB
■ Secrets & .env	Secure credential management
■ Bonus Services	Redis, FTP, Adminer, static site

Covers ALL mandatory requirements + evaluation checklist + bonus services

Table of Contents

Chapter 1 — Concepts & Theory

- Docker vs Virtual Machines
- Docker Images & Containers
- Docker Compose
- Networking in Docker
- Volumes & Data Persistence
- Secrets vs Environment Variables
- TLS / SSL Certificates
- NGINX as Reverse Proxy
- PHP-FPM
- MariaDB

Chapter 2 — Project Architecture

- Directory Structure
- Service Overview Diagram
- Data Flow
- Security Design

Chapter 3 — Environment Setup

- Virtual Machine Setup
- Installing Docker & Docker Compose
- Project Skeleton

Chapter 4 — MariaDB Container

- Dockerfile
- Entrypoint Script
- Configuration

Chapter 5 — WordPress + PHP-FPM Container

- Dockerfile
- wp-config & Entrypoint
- WP-CLI Setup

Chapter 6 — NGINX Container

- Dockerfile
- TLS Certificate Generation
- NGINX Configuration

Chapter 7 — Docker Compose

- docker-compose.yml
- Secrets Setup
- .env File
- Volumes & Networks

Chapter 8 — Makefile

- Targets & Commands

Chapter 9 — Required Documentation

- README.md
- USER_DOC.md
- DEV_DOC.md

Chapter 10 — Validation Checklist

- Evaluation Sheet Walk-Through
- Common Pitfalls
- Live Defense Tips

Chapter 11 — Bonus Services

- Redis Cache
- FTP Server
- Static Website
- Adminer
- Custom Service

Chapter 12 — Full File Reference

- All Files Listed

1 Concepts & Theory

Before writing a single line of configuration, you must understand the technologies involved. The evaluation requires you to explain these concepts in simple terms. This chapter is your reference — read it carefully.

1.1 Docker vs Virtual Machines

Virtual Machine (VM)	Docker Container
• Runs a full OS (kernel + userland) on top of a hypervisor	• Shares the host kernel — only packages the application + deps
• Each VM needs its own OS installation — heavy on disk & RAM	• Lightweight: typically 10–200 MB vs 2–20 GB for a VM
• Boot time: minutes	• Start time: milliseconds
• Strong isolation (separate kernel)	• Process-level isolation (namespaces, cgroups)
• Hypervisors: VirtualBox, VMware, KVM	• Engine: Docker daemon
• Good for: running completely different OSes side-by-side	• Good for: microservices, reproducible environments, CI/CD

In Inception you run Docker *inside* a virtual machine. This respects the subject rule (work must be done in a VM) while still using Docker containers for isolation of each service.

1.2 Docker Images vs Containers

An **image** is a read-only template — a snapshot of a filesystem. A **container** is a running instance of an image. Think of an image as a class, and a container as an object/instance of that class.

- **Dockerfile** → defines how to build an image (layer by layer)
- **docker build** → creates an image from a Dockerfile
- **docker run** → starts a container from an image
- Each RUN instruction in a Dockerfile adds a new *layer*; layers are cached and reused

■ In Inception, pulling pre-built images from DockerHub is **FORBIDDEN** (except Alpine/Debian base images). You must write your own Dockerfiles for every service.

1.3 Docker Compose

Docker Compose is a tool that lets you define and run multi-container applications using a YAML file (**docker-compose.yml**). Instead of running multiple *docker run* commands with many flags, you describe everything declaratively and start it all with one command.

```
# Start everything (build if needed)
docker compose up --build -d

# Stop everything
docker compose down

# View logs
docker compose logs -f

# List running containers
docker compose ps
```

Without docker compose: each container is started manually with *docker run* and must be individually linked, networked, and managed.

With docker compose: one file describes the entire stack — services, networks, volumes, environment variables — and all containers are managed together.

1.4 Docker Networking

Docker Network (bridge/custom)	Host Network (--network host)
Containers get their own virtual network interfaces	Container shares the host's network namespace
Services communicate by container name (DNS)	No isolation — container sees all host ports
Isolated from host network — only exposed ports are reachable from outside	Security risk; not suitable for production
Required by Inception	FORBIDDEN in Inception ('network: host' is prohibited)
Example: <code>nginx</code> → <code>wordpress:9000</code> (via container name)	Also forbidden: '--link' and 'links:'

Inception requires a **custom bridge network** defined in `docker-compose.yml`. Containers in the same network can reach each other by service name. Only NGINX exposes a port (443) to the

outside world.

1.5 Volumes & Data Persistence

Named Docker Volume	Bind Mount
• Managed entirely by Docker	• You specify an exact host path
• Data lives in <code>/var/lib/docker/volumes/</code> on the host	• example: <code>- /home/user/data:/var/lib/mysql</code>
• Docker controls the path	• Host path must exist; file permissions can cause issues
• Can be inspected with <code>docker volume inspect</code>	• Good for development (live code reloading)
• REQUIRED by Inception for WordPress and database data	• FORBIDDEN for the main volumes in Inception

■ Named volumes in Inception must store data at `/home//data` on the host. This is configured via the `'driver_opts'` in `docker-compose.yml`, not a bind mount.

1.6 Secrets vs Environment Variables

Docker Secrets	Environment Variables (.env)
• Stored as files on disk, mounted inside container at <code>/run/secrets/</code>	• Stored in <code>.env</code> file, passed as environment variables to containers
• Not visible in <code>'docker inspect'</code> output or environment	• Visible in <code>'docker inspect'</code> — less secure
• Best for: passwords, API keys, TLS keys	• <code>.env</code> file must NEVER be committed to Git
• Strongly recommended by Inception for confidential data	• Required by Inception for non-secret config (domain name, usernames, etc.)

■ Any password, API key, or credential found in your Git repository (outside properly configured secrets files) results in an automatic project failure (grade = 0).

1.7 TLS / SSL Certificates

TLS (Transport Layer Security) encrypts data between the browser and your server. HTTPS uses TLS. You need a certificate to prove the server's identity.

- **TLS 1.2 / 1.3**: the two modern, secure versions. Inception requires one of these.
- **Self-signed certificate**: generated locally — not trusted by public CAs, but valid for the project.
The browser will show a warning, which is expected.
- **openssl**: the tool used to generate the certificate and private key.

```
# Generate a self-signed cert (done inside the NGINX Dockerfile or entrypoint)
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/nginx.key \
  -out /etc/ssl/certs/nginx.crt \
  -subj "/C=BE/ST=Brussels/L=Brussels/O=42/CN=login.42.fr"
```

1.8 NGINX as Reverse Proxy

NGINX in Inception acts as a **reverse proxy** and **TLS terminator**. It is the *only* entry point from the internet (port 443). All HTTPS requests arrive at NGINX, which forwards PHP requests to WordPress via FastCGI (port 9000).

- Listens on port 443 (HTTPS/TLS) only
- Decrypts TLS traffic and proxies to WordPress container on port 9000 via FastCGI
- Serves static files directly from the WordPress volume
- Port 80 (HTTP) must NOT be accessible from outside

1.9 PHP-FPM

PHP-FPM (FastCGI Process Manager) is a way to run PHP scripts. Unlike the classic Apache+mod_php setup, PHP-FPM runs as a separate process that web servers (NGINX) communicate with via the FastCGI protocol.

- NGINX does NOT run PHP — it delegates to php-fpm via FastCGI on port 9000
- php-fpm processes PHP scripts and returns the HTML result to NGINX
- In Inception, the WordPress container runs php-fpm with NO nginx

1.10 MariaDB

MariaDB is an open-source relational database (a MySQL fork). WordPress stores all its content — posts, users, settings — in a MariaDB database. In Inception, MariaDB runs in its own container with a dedicated volume for data persistence.

- Default port: 3306 (only accessible within the Docker network)
- Two users required: one admin (username must NOT contain 'admin'/'administrator') and one regular user
- Data stored in a named Docker volume mounted at /home/login/data/db

1.11 PID 1 & Daemon Best Practices

In a container, the first process started becomes PID 1. This process is special: it receives signals (like SIGTERM for graceful shutdown). If you use a shell script as your entrypoint, the shell becomes PID 1 and may swallow signals instead of passing them to your app.

- Use **exec** at the end of entrypoint scripts to replace the shell with your daemon
- Example: **exec mysqld_safe** instead of just **mysqld_safe**
- NEVER use: `tail -f`, `bash`, `sleep infinity`, `while true` — these are forbidden in Inception
- Run services in the FOREGROUND (as PID 1) so Docker can manage their lifecycle

```
#!/bin/sh
# Good entrypoint pattern
# ... setup commands ...
exec mysqld_safe # replaces shell as PID 1 – handles signals correctly
```


2 Project Architecture

2.1 Directory Structure

Your repository must follow this exact layout:

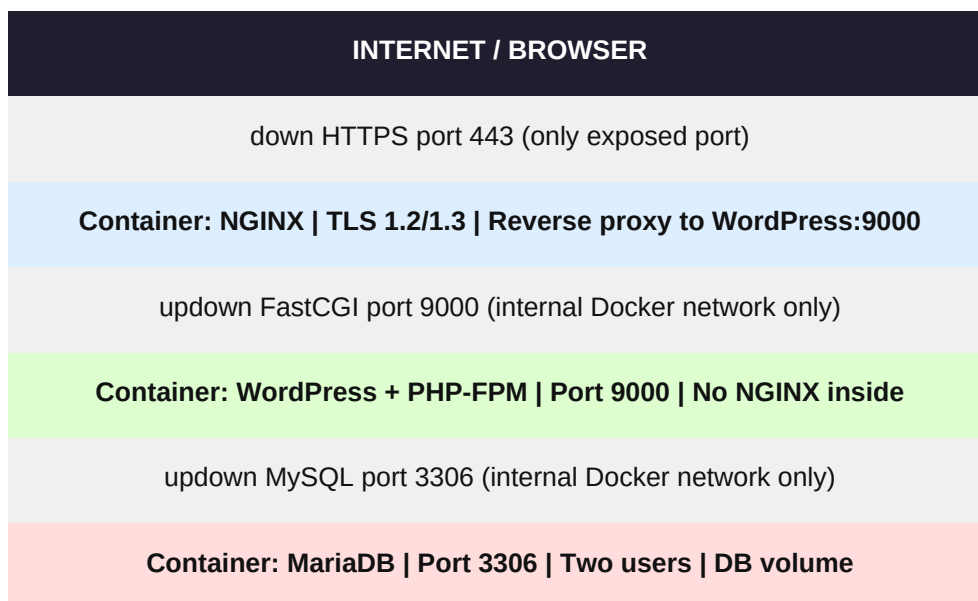
```

project_root/      ← Git root
■■■■ Makefile      ← builds & orchestrates everything
■■■■ README.md     ← mandatory (see Chapter 9)
■■■■ USER_DOC.md   ← mandatory (end-user guide)
■■■■ DEV_DOC.md    ← mandatory (developer guide)
■■■■ secrets/      ← credentials (git-ignored!)
■   ■■■■ credentials.txt ← WordPress admin credentials
■   ■■■■ db_password.txt ← MariaDB user password
■   ■■■■ db_root_password.txt
■■■■ srcs/
■■■■ .env          ← environment variables (git-ignored!)
■■■■ docker-compose.yml
■■■■ requirements/
■   ■■■■ nginx/
■   ■■■■ Dockerfile
■   ■■■■ .dockerignore
■   ■■■■ conf/
■   ■   ■■■■ nginx.conf
■   ■■■■ tools/
■       ■■■■ generate_cert.sh (optional)
■■■■ wordpress/
■   ■■■■ Dockerfile
■   ■■■■ .dockerignore
■   ■■■■ conf/
■   ■   ■■■■ www.conf
■   ■■■■ tools/
■       ■■■■ wp_setup.sh
■■■■ mariadb/
■   ■■■■ Dockerfile
■   ■■■■ .dockerignore
■   ■■■■ conf/
■   ■   ■■■■ my.cnf
■   ■■■■ tools/
■       ■■■■ db_init.sh
■■■■ bonus/      ← (optional bonus services)

```

2.2 Service Architecture Overview

The infrastructure consists of three mandatory containers connected through a custom Docker network. Only NGINX is exposed to the outside on port 443.



2.3 Volumes

Two named Docker volumes are required:

- **wordpress_data** — WordPress files. Mounted in NGINX and WordPress containers. Stored at `/home/login/data/wordpress` on the VM.
- **db_data** — MariaDB database files. Mounted only in the MariaDB container. Stored at `/home/login/data/db` on the VM.

■■ Both volumes persist across container restarts and reboots. This is tested in the evaluation.

2.4 Security Design

- Only port 443 is exposed to the host machine (via NGINX)
- Port 3306 (MariaDB) and 9000 (PHP-FPM) are internal to the Docker network
- Passwords are stored in Docker secrets (files in `/run/secrets/` inside containers)
- No password in any Dockerfile
- The 'latest' tag is forbidden — always pin a specific version
- `.env` and `secrets/` are listed in `.gitignore`

3 Environment Setup

3.1 Virtual Machine Setup

The project must run inside a Virtual Machine. A common choice at 42 is Debian or Ubuntu LTS on VirtualBox or UTM (Apple Silicon).

1. Download Debian 12 ISO from <https://www.debian.org>
2. Create a VM with at least 2 CPU cores, 4 GB RAM, 30 GB disk
3. Install a minimal Debian system (no GUI needed)
4. Set up SSH access for convenience
5. Edit `/etc/hosts` to map your domain to 127.0.0.1:

```
# /etc/hosts inside your VM
127.0.0.1    localhost
127.0.0.1    yourlogin.42.fr
```

3.2 Installing Docker & Docker Compose

```
# Install dependencies
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg \
    | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/debian \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine and Compose plugin
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin

# Allow your user to run Docker without sudo
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker compose version
```

3.3 Creating the Project Skeleton

```
# Replace 'yourlogin' with your actual 42 login
LOGIN=yourlogin

# Create directory structure
mkdir -p srcs/requirements/{nginx,wordpress,mariadb}/{conf,tools}
mkdir -p srcs/requirements/bonus
mkdir -p secrets

# Create data directories on the VM
mkdir -p /home/$LOGIN/data/{wordpress,db}

# Create placeholder files
touch Makefile README.md USER_DOC.md DEV_DOC.md
touch srcs/docker-compose.yml srcs/.env
touch secrets/credentials.txt secrets/db_password.txt secrets/db_root_password.txt

# Create .gitignore
cat > .gitignore << 'EOF'
srcs/.env
secrets/
EOF
```

3.4 Populate the .env file

The .env file is loaded by docker-compose and provides environment variables to containers. It must NEVER be committed to Git.

```
# srcs/.env
DOMAIN_NAME=yourlogin.42.fr

# MariaDB
MYSQL_DATABASE=wordpress
MYSQL_USER=wpuser
MYSQL_USER_PASSWORD=changeme_user # will be overridden by secret in production
MYSQL_ROOT_PASSWORD=changeme_root # will be overridden by secret in production

# WordPress
WP_ADMIN_USER=wpadmin42 # Must NOT contain 'admin' or 'administrator'
WP_ADMIN_EMAIL=admin@yourlogin.42.fr
WP_USER=wpeditor
WP_USER_EMAIL=editor@yourlogin.42.fr

# Paths
DATA_PATH=/home/yourlogin/data
```

3.5 Populate the secrets files

```
# secrets/db_password.txt
StrongPass_WordUser42!

# secrets/db_root_password.txt
StrongPass_Root42!

# secrets/credentials.txt (WordPress admin password)
StrongPass_WpAdmin42!
```

■ Each file should contain **ONLY** the password, with no trailing newline issues. Use **printf** instead of **echo** to avoid newlines.

4 MariaDB Container

MariaDB is the database backend. It must run in its own container with no NGINX. Data must persist in the named volume even after container restarts.

4.1 Dockerfile

```
# srcs/requirements/mariadb/Dockerfile
FROM debian:bookworm                # penultimate stable = bookworm (Debian 12)

# Install MariaDB server
RUN apt-get update && apt-get install -y --no-install-recommends \
    mariadb-server \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Copy configuration file
COPY conf/my.cnf /etc/mysql/conf.d/my.cnf

# Copy initialization script
COPY tools/db_init.sh /usr/local/bin/db_init.sh
RUN chmod +x /usr/local/bin/db_init.sh

# Expose MySQL port (internal network only)
EXPOSE 3306

ENTRYPOINT ["/usr/local/bin/db_init.sh"]
```

■ ■ Use 'debian:bookworm' (Debian 12, the penultimate stable at the time of writing). The penultimate means the **SECOND-MOST-RECENT** stable release. Check which is current at exam time — it could be 'debian:bullseye' or 'alpine:3.18', etc.

4.2 MariaDB Configuration (my.cnf)

```
# srcs/requirements/mariadb/conf/my.cnf
[mysqld]
# Listen on all interfaces so WordPress container can connect
bind-address = 0.0.0.0
port = 3306

# Data directory
datadir = /var/lib/mysql

# Disable DNS lookups for performance in containers
skip-name-resolve

[client]
default-character-set = utf8mb4

[mysqld_safe]
log-error = /var/log/mysql/error.log
pid-file = /var/run/mysqld/mysqld.pid
```

4.3 Initialization Script (db_init.sh)

This script initializes the database on first run, creates the database, users, and grants permissions, then starts MariaDB in the foreground as PID 1.

```
#!/bin/sh
# srcs/requirements/mariadb/tools/db_init.sh
set -e

# Read passwords from Docker secrets
MYSQL_ROOT_PASSWORD=$(cat /run/secrets/db_root_password)
MYSQL_USER_PASSWORD=$(cat /run/secrets/db_password)

# Create runtime directories
mkdir -p /var/run/mysqld /var/log/mysql
chown -R mysql:mysql /var/run/mysqld /var/log/mysql /var/lib/mysql

# Initialize data directory only on first run
if [ ! -d "/var/lib/mysql/mysql" ]; then
    echo ">> First run: initializing MariaDB data directory..."

    # Initialize the database
    mysqld --initialize-insecure --user=mysql --datadir=/var/lib/mysql

    # Start temporary server to run setup
    mysqld --user=mysql --skip-networking &
    TEMP_PID=$!

    # Wait for MariaDB to be ready
    while ! mysqladmin ping --silent 2>/dev/null; do
        sleep 0.5
    done

    # Create database and users
    mysql -u root << EOSQL
    CREATE DATABASE IF NOT EXISTS ${MYSQL_DATABASE};
    CREATE USER IF NOT EXISTS '${MYSQL_USER}'@'%' IDENTIFIED BY '${MYSQL_USER_PASSWORD}';
    GRANT ALL PRIVILEGES ON ${MYSQL_DATABASE}.* TO '${MYSQL_USER}'@'%';
    ALTER USER 'root'@'localhost' IDENTIFIED BY '${MYSQL_ROOT_PASSWORD}';
    FLUSH PRIVILEGES;
EOSQL

    # Stop temporary server
    kill $TEMP_PID
    wait $TEMP_PID 2>/dev/null || true
    echo ">> MariaDB initialization complete."
fi

# Start MariaDB in foreground as PID 1
exec mysqld_safe --user=mysql
```

■ The 'exec' at the end is critical: it replaces the shell with mysqld_safe, making it PID 1 and allowing Docker to send signals correctly (e.g., SIGTERM on docker stop).

5

WordPress + PHP-FPM Container

WordPress is the CMS. PHP-FPM handles PHP execution. NGINX (in another container) forwards PHP requests here via FastCGI on port 9000.

5.1 Dockerfile

```
# srcs/requirements/wordpress/Dockerfile
FROM debian:bookworm

# Install PHP-FPM, WordPress dependencies, and WP-CLI prerequisites
RUN apt-get update && apt-get install -y --no-install-recommends \
    php8.2-fpm \
    php8.2-mysql \
    php8.2-curl \
    php8.2-gd \
    php8.2-intl \
    php8.2-zip \
    php8.2-xml \
    php8.2-mbstring \
    wget \
    curl \
    mariadb-client \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Install WP-CLI (WordPress command-line tool)
RUN curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar \
    && chmod +x wp-cli.phar \
    && mv wp-cli.phar /usr/local/bin/wp

# Copy PHP-FPM pool configuration
COPY conf/www.conf /etc/php/8.2/fpm/pool.d/www.conf

# Copy WordPress setup script
COPY tools/wp_setup.sh /usr/local/bin/wp_setup.sh
RUN chmod +x /usr/local/bin/wp_setup.sh

# WordPress files will be served from this directory
WORKDIR /var/www/html

# PHP-FPM listens on port 9000
EXPOSE 9000

ENTRYPOINT ["/usr/local/bin/wp_setup.sh"]
```

5.2 PHP-FPM Pool Configuration (www.conf)

```
# srcs/requirements/wordpress/conf/www.conf
[www]
user = www-data
group = www-data

# Listen on TCP port 9000 (NGINX connects here via Docker network)
listen = 0.0.0.0:9000

listen.owner = www-data
listen.group = www-data

pm = dynamic
pm.max_children = 5
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3

# Environment variables passed to PHP
clear_env = no
```

5.3 WordPress Setup Script (wp_setup.sh)

```
#!/bin/sh
# srcs/requirements/wordpress/tools/wp_setup.sh
set -e

WP_PATH=/var/www/html

# Read secrets
DB_PASSWORD=$(cat /run/secrets/db_password)
WP_ADMIN_PASSWORD=$(cat /run/secrets/credentials)

# Wait for MariaDB to be ready
echo ">> Waiting for MariaDB..."
while ! mysqladmin ping -h"${MYSQL_HOST}" -u"${MYSQL_USER}" \
    -p"${DB_PASSWORD}" --silent 2>/dev/null; do
    sleep 2
done
echo ">> MariaDB is ready."

# Download and configure WordPress only if not already installed
if [ ! -f "${WP_PATH}/wp-config.php" ]; then
    echo ">> Installing WordPress..."

    # Download WordPress
    wp core download --path=${WP_PATH} --allow-root

    # Create wp-config.php
    wp config create \
        --path=${WP_PATH} \
        --dbname=${MYSQL_DATABASE} \
        --dbuser=${MYSQL_USER} \
        --dbpass=${DB_PASSWORD} \
        --dbhost=mariadb:3306 \
        --allow-root

    # Install WordPress core
    wp core install \
        --path=${WP_PATH} \
        --url=https://${DOMAIN_NAME} \
        --title="Inception" \
        --admin_user=${WP_ADMIN_USER} \
        --admin_password=${WP_ADMIN_PASSWORD} \
        --admin_email=${WP_ADMIN_EMAIL} \
        --skip-email \
        --allow-root

    # Create a second (non-admin) user
    wp user create \
        ${WP_USER} ${WP_USER_EMAIL} \
        --role=editor \
        --user_pass=${WP_ADMIN_PASSWORD} \
        --path=${WP_PATH} \
        --allow-root

    # Fix permissions
    chown -R www-data:www-data ${WP_PATH}
fi

echo ">> Starting PHP-FPM..."
# Start php-fpm in foreground as PID 1
# The '-F' flag keeps php-fpm in the foreground
exec php-fpm8.2 -F
```

■■ MYSQL_HOST should be 'mariadb' — the service name in docker-compose.yml. Docker's built-in DNS resolves container names within the same network.

6 NGINX Container

NGINX is the sole entry point. It terminates TLS and proxies requests to WordPress. Port 443 only — port 80 must not be accessible.

6.1 Dockerfile

```
# srcs/requirements/nginx/Dockerfile
FROM debian:bookworm

# Install NGINX and OpenSSL
RUN apt-get update && apt-get install -y --no-install-recommends \
    nginx \
    openssl \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Copy NGINX configuration
COPY conf/nginx.conf /etc/nginx/sites-available/default

# Copy certificate generation script
COPY tools/generate_cert.sh /usr/local/bin/generate_cert.sh
RUN chmod +x /usr/local/bin/generate_cert.sh

# Generate TLS certificate at build time
RUN /usr/local/bin/generate_cert.sh

# Expose HTTPS port only
EXPOSE 443

# Start NGINX in foreground
CMD ["nginx", "-g", "daemon off;"]
```

6.2 Certificate Generation Script

```
#!/bin/sh
# srcs/requirements/nginx/tools/generate_cert.sh

CERT_DIR=/etc/ssl/certs
KEY_DIR=/etc/ssl/private

mkdir -p ${CERT_DIR} ${KEY_DIR}

openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout ${KEY_DIR}/nginx.key \
    -out ${CERT_DIR}/nginx.crt \
    -subj "/C=BE/ST=Brussels/L=Brussels/O=42School/CN=yourlogin.42.fr"

echo ">> TLS certificate generated."
```

■ You can also generate the certificate inside an entrypoint script at runtime. Generating at build time (as shown) is simpler and faster.

6.3 NGINX Configuration (nginx.conf)

```
# srcs/requirements/nginx/conf/nginx.conf
server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name yourlogin.42.fr;

    # TLS configuration — only TLSv1.2 and TLSv1.3
    ssl_certificate      /etc/ssl/certs/nginx.crt;
    ssl_certificate_key  /etc/ssl/private/nginx.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers          HIGH:!aNULL:!MD5;

    # WordPress root
    root /var/www/html;
    index index.php index.html;

    # Serve requests — try file, then directory, then PHP fallback
    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    # Pass PHP scripts to WordPress container via FastCGI
    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass wordpress:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param PHP_VALUE "upload_max_filesize = 10M;";
    }

    # Deny access to hidden files
    location ~ /\. {
        deny all;
    }
}

# No HTTP server block — port 80 is not exposed
```

■ **Do NOT add a server block for port 80. The project explicitly requires HTTPS only. Any redirect from 80 to 443 is also prohibited since port 80 must simply not work.**

The key line is **fastcgi_pass wordpress:9000** — 'wordpress' is the service name in docker-compose.yml. Docker DNS resolves it to the WordPress container IP within the custom network.

7 Docker Compose

7.1 The docker-compose.yml File

```
# srcs/docker-compose.yml  
version: '3.8'  
  
services:  
  
# ■■■ MariaDB ■■■  
mariadb:  
  build: ./requirements/mariadb  
  image: mariadb # image name = service name (required)  
  container_name: mariadb  
  restart: unless-stopped # auto-restart on crash  
  env_file: .env  
  secrets:  
    - db_password  
    - db_root_password  
  volumes:  
    - db_data:/var/lib/mysql  
  networks:  
    - inception_net  
  expose:  
    - "3306" # internal only, not published to host  
  
# ■■■ WordPress ■■■  
wordpress:  
  build: ./requirements/wordpress  
  image: wordpress  
  container_name: wordpress  
  restart: unless-stopped  
  env_file: .env  
  secrets:  
    - db_password  
    - credentials  
  environment:  
    MYSQL_HOST: mariadb # use service name for DNS resolution  
  volumes:  
    - wordpress_data:/var/www/html  
  networks:  
    - inception_net  
  expose:  
    - "9000"  
  depends_on:  
    - mariadb
```

```
# ■■■■ NGINX ■■■■
nginx:
  build: ./requirements/nginx
  image: nginx
  container_name: nginx
  restart: unless-stopped
  env_file: .env
  volumes:
    - wordpress_data:/var/www/html # shares WP files with WordPress
  ports:
    - "443:443" # only exposed port to the host
  networks:
    - inception_net
  depends_on:
    - wordpress

# ■■■■ Secrets ■■■■
secrets:
  db_password:
    file: ../secrets/db_password.txt
  db_root_password:
    file: ../secrets/db_root_password.txt
  credentials:
    file: ../secrets/credentials.txt

# ■■■■ Networks ■■■■
networks:
  inception_net:
    driver: bridge

# ■■■■ Volumes ■■■■
volumes:
  db_data:
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /home/yourlogin/data/db

  wordpress_data:
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /home/yourlogin/data/wordpress
```

■ The volumes use `driver_opts` to pin the storage path to `/home/login/data`. The directories `/home/yourlogin/data/db` and `/home/yourlogin/data/wordpress` MUST exist on the VM before running `docker compose up`. Create them in your `Makefile`.

Key points about this docker-compose.yml:

- No 'network: host' anywhere — custom bridge network only
- No '--link' — containers find each other by service name via Docker DNS
- 'restart: unless-stopped' — containers restart automatically on crash
- 'image: ' must match the service name — this is a subject requirement
- Secrets are mounted as files at /run/secrets/ inside containers
- Only NGINX publishes a port (443) to the host — others use 'expose' (internal only)

8 Makefile

The Makefile must be at the root of your repository. It orchestrates building and running the entire infrastructure.

```
# Makefile — at project root
LOGIN := yourlogin
DATA := /home/$(LOGIN)/data
COMPOSE := docker compose -f srcs/docker-compose.yml

.PHONY: all build up down clean fclean re logs ps

all: build up

# Create data directories, build images, start containers
build:
  ■@mkdir -p $(DATA)/wordpress $(DATA)/db
  ■$(COMPOSE) build

up:
  ■$(COMPOSE) up -d

# Alias: build + up
start: build up

# Stop and remove containers (keep volumes and images)
down:
  ■$(COMPOSE) down

# Stop + remove containers and images
clean: down
  ■docker image prune -af

# Full clean: containers + images + volumes + data
fclean: down
  ■docker system prune -af --volumes
  ■sudo rm -rf $(DATA)/wordpress/* $(DATA)/db/*

# Rebuild everything from scratch
re: fclean all

# Useful shortcuts
logs:
  ■$(COMPOSE) logs -f

ps:
  ■$(COMPOSE) ps

# Evaluation helper: reset Docker state completely
eval-clean:
  ■docker stop $$($(docker ps -qa) 2>/dev/null || true)
  ■docker rm $$($(docker ps -qa) 2>/dev/null || true)
  ■docker rmi -f $$($(docker images -qa) 2>/dev/null || true)
  ■docker volume rm $$($(docker volume ls -q) 2>/dev/null || true)
  ■docker network rm $$($(docker network ls -q) 2>/dev/null || true)
```

■ ■ The evaluation sheet requires running this cleanup command before starting: 'docker stop \$(docker ps -qa); docker rm \$(docker ps -qa); docker rmi -f \$(docker images -qa); docker volume rm \$(docker volume ls -q); docker network rm \$(docker network ls -q) 2>/dev/null'. The eval-clean target handles this.

9 Required Documentation

Three mandatory documentation files are required at the root of your repository. Missing or empty files cause immediate failure during evaluation.

9.1 README.md

```
<!-- README.md -->
*This project has been created as part of the 42 curriculum by yourlogin.*

# Inception

## Description

Inception is a system administration project that sets up a small containerized
infrastructure using Docker and Docker Compose. It consists of three services:
- **NGINX**: HTTPS reverse proxy (TLSv1.3, port 443)
- **WordPress + PHP-FPM**: CMS application
- **MariaDB**: relational database

### Design Choices

**Virtual Machines vs Docker**
VMS virtualize an entire OS (heavy, slow to start). Docker containers share the
host kernel and only package application dependencies – lightweight and fast.

**Secrets vs Environment Variables**
Docker secrets are mounted as files at `/run/secrets/` and never appear in
`docker inspect` output. Environment variables (.env) are visible in inspections
and are used for non-sensitive configuration only.

**Docker Network vs Host Network**
A custom bridge network isolates containers from the host. Only NGINX exposes
port 443. Host network bypasses isolation and is forbidden in this project.

**Docker Volumes vs Bind Mounts**
Named volumes are managed by Docker; bind mounts require specific host paths.
Named volumes are required for persistent data. Bind mounts are forbidden for
the main data volumes in this project.

## Instructions
```

```
### Prerequisites
- Linux VM (Debian recommended)
- Docker & Docker Compose installed
- yourlogin.42.fr added to /etc/hosts -> 127.0.0.1

### Setup
```bash
Clone the repo
git clone <repo-url> && cd inception

Create secrets (not in git)
echo 'StrongPass42!' > secrets/db_password.txt
echo 'StrongRootPass42!' > secrets/db_root_password.txt
echo 'WPAdminPass42!' > secrets/credentials.txt

Edit srcs/.env with your login

Build and start
make
```

Access: https://yourlogin.42.fr (accept the self-signed certificate warning)

## Resources

- [Docker documentation](https://docs.docker.com)
- [NGINX FastCGI guide](https://nginx.org/en/docs/http/nginx_http_fastcgi_module.html)
- [MariaDB documentation](https://mariadb.com/kb/en/documentation/)
- [WordPress WP-CLI](https://wp-cli.org)
- [PHP-FPM documentation](https://www.php.net/manual/en/install.fpm.configuration.php)

**AI Usage**: AI was used to help draft entrypoint scripts and NGINX configuration.
All generated code was reviewed, tested, and understood before inclusion.
```

9.2 USER_DOC.md

```

<!-- USER_DOC.md -->
# User Documentation – Inception

## Services Provided

Service	Description	Access
WordPress	Blog/CMS website	https://yourlogin.42.fr
MariaDB	Database (internal only)	port 3306, Docker net only
NGINX	HTTPS reverse proxy	port 443

## Starting and Stopping the Project

```bash
Start
make

Stop (keep data)
make down

Stop and clean everything
make fclean
```

## Accessing the Website

1. Open your browser and go to: https://yourlogin.42.fr
2. Accept the self-signed certificate warning (click 'Advanced' → 'Proceed')
3. The WordPress site will load

## Administration Panel

1. Go to https://yourlogin.42.fr/wp-admin
2. Log in with the admin credentials from secrets/credentials.txt
   Username: (see WP_ADMIN_USER in srcs/.env)
   Password: (see secrets/credentials.txt)

## Managing Credentials

Credentials are stored in the `secrets/` folder (not in Git):
- `secrets/credentials.txt` – WordPress admin password
- `secrets/db_password.txt` – MariaDB regular user password
- `secrets/db_root_password.txt` – MariaDB root password

## Checking Service Status

```bash
View running containers
docker compose -f srcs/docker-compose.yml ps

View logs for a specific service
docker compose -f srcs/docker-compose.yml logs nginx
docker compose -f srcs/docker-compose.yml logs wordpress
docker compose -f srcs/docker-compose.yml logs mariadb

Check volumes
docker volume ls
docker volume inspect srcs_db_data
```

```

9.3 DEV_DOC.md

```

<!-- DEV_DOC.md -->
# Developer Documentation – Inception

## Prerequisites

- Linux VM (Debian 12 recommended)
- Docker CE + Docker Compose plugin
- At least 4 GB RAM, 20 GB disk
- yourlogin.42.fr in /etc/hosts

## Setup From Scratch

```bash
1. Clone
git clone <repo-url> inception && cd inception

2. Create secrets (never committed to git)
echo -n 'StrongPass42!' > secrets/db_password.txt
echo -n 'StrongRootPass42!' > secrets/db_root_password.txt
echo -n 'WPAdminPass42!' > secrets/credentials.txt

3. Configure .env
cp srcs/.env.example srcs/.env # if you provide an example
Edit srcs/.env with your login and settings

4. Create data directories
mkdir -p /home/yourlogin/data/{wordpress,db}

5. Build and start
make
```

## Makefile Targets

| Target | Action |

```

```
|-----|-----|
| make    | Build images and start containers |
| make down| Stop and remove containers (data preserved) |
| make clean| Stop, remove containers and images |
| make fclean| Full clean including volumes and data |
| make re  | Rebuild everything from scratch |
| make logs| Follow container logs |
| make ps  | Show container status |
```

Key Docker Commands

```
```bash
```

```
Enter a running container
```

```
docker exec -it mariadb sh
```

```
docker exec -it wordpress bash
```

```
docker exec -it nginx bash
```

```
Connect to MariaDB
```

```
docker exec -it mariadb mysql -u root -p
```

```
Inspect a volume
```

```
docker volume inspect srcs_db_data
```

```
View data on host
```

```
ls -la /home/yourlogin/data/
```

```
```
```

Data Persistence

Data is stored on the VM at:

- `/home/yourlogin/data/db/`` – MariaDB database files

- `/home/yourlogin/data/wordpress/`` – WordPress files

These directories are mapped to named Docker volumes via `driver_opts` in `docker-compose.yml`. Data survives container restarts and VM reboots.

10

Validation Checklist & Defense

This chapter walks through the evaluation sheet point by point, telling you exactly what the evaluator will check and how to verify it yourself.

10.1 Pre-Defense: Self-Evaluation Reset

Run this before your evaluation begins:

```
docker stop $(docker ps -qa) 2>/dev/null; \
docker rm $(docker ps -qa) 2>/dev/null; \
docker rmi -f $(docker images -qa) 2>/dev/null; \
docker volume rm $(docker volume ls -q) 2>/dev/null; \
docker network rm $(docker network ls -q) 2>/dev/null
```

10.2 Evaluation Sheet Checklist

Git Repository

- No credentials in git repo (outside secrets/)
- .env and secrets/ are in .gitignore
- Correct directory structure (srcs/, Makefile at root)

General Instructions

- srcs/ folder at root of repo
- Makefile at root of repo
- No 'network: host' in docker-compose.yml
- No 'links:' in docker-compose.yml
- Networks section present in docker-compose.yml
- No '--link' in any script or Makefile
- No 'tail -f', 'bash', 'sleep infinity', 'while true' in Dockerfiles ENTRYPOINT
- Containers built from Alpine or Debian penultimate stable
- Makefile successfully builds everything with 'make'

README & Documentation

- README.md present at repo root
- First line: italicized 'This project has been created as part of the 42 curriculum by '
- Sections: Description, Instructions, Resources (with AI usage described)
- Project description includes VM vs Docker, Secrets vs Env, Docker Network vs Host, Volumes vs Bind Mounts
- USER_DOC.md present and non-empty
- DEV_DOC.md present and non-empty

Simple Setup

- Port 443 accessible via <https://login.42.fr>
- Port 80 NOT accessible (connection refused or timeout)
- SSL/TLS certificate in use (may show browser warning — that's OK)
- WordPress site loads (NOT the installation wizard)

NGINX Container

- Dockerfile exists in nginx/
- Container appears in 'docker compose ps'
- TLS v1.2 or v1.3 certificate in use
- <https://login.42.fr> shows WordPress site
- <http://login.42.fr> does NOT connect

WordPress Container

- Dockerfile exists in wordpress/
- No NGINX in the WordPress Dockerfile
- Container appears in 'docker compose ps'
- Volume mounted: 'docker volume inspect' shows /home/login/data/
- Can add a comment as a WordPress user
- Admin username does NOT contain 'admin' or 'administrator'
- Can log into wp-admin and edit pages

MariaDB Container

- Dockerfile exists in mariadb/
- No NGINX in the MariaDB Dockerfile
- Container appears in 'docker compose ps'
- Volume mounted: 'docker volume inspect' shows /home/login/data/
- Can log into the database (evaluator may ask how)

Persistence

- Reboot the VM, restart containers with 'make', verify WordPress works
- Changes made before reboot (comments, edits) are still present

Configuration Modification (Live Exercise)

- Evaluator asks you to change a port or config
- You rebuild and restart: 'make re' or targeted rebuild
- Service works with the new configuration

10.3 Concepts You Must Be Able to Explain

The evaluator will ask you to explain these in simple terms:

Q: How Docker and docker compose work

→ Docker packages apps + dependencies into containers sharing the host kernel. Docker Compose orchestrates multiple containers via a YAML file — one command starts the whole stack.

Q: Docker image vs container

→ Image = read-only template (like a class). Container = running instance of an image (like an object).

Q: Docker with compose vs without compose

→ Without compose: each container started manually with docker run + many flags, hard to manage. With compose: one file, one command, networks and volumes auto-created.

Q: Docker vs VM

→ VMs virtualize hardware + run a full OS. Containers share the host kernel, only package the app. Containers are faster, lighter, and more portable.

Q: Why the directory structure

→ Each service has its own folder with a Dockerfile for clear separation. srcs/ separates config from the Makefile. secrets/ keeps sensitive data out of the image build context.

Q: How to log into MariaDB

→ docker exec -it mariadb mysql -u wpuser -p (enter password from secrets/db_password.txt)

Q: What a Docker network does

→ Creates an isolated virtual network so containers can communicate by service name. Prevents external access to internal ports like 3306 and 9000.

Q: Secrets vs .env

→ Secrets are files mounted inside the container at `/run/secrets/` — not visible in `'docker inspect'`. `.env` variables ARE visible in inspect output. Use secrets for passwords, `.env` for non-sensitive config.

Q: Docker volumes vs bind mounts

→ Named volumes: Docker manages the path, data at `/var/lib/docker/volumes/`. Bind mounts: you specify the exact host path. Inception requires named volumes (with pinned host path via `driver_opts`).

10.4 Common Pitfalls

| | |
|---|---|
| ■ Admin username contains 'admin' | ■ Use a completely different username like 'wadmin42' or 'sysop' |
| ■ 'latest' tag used in Dockerfile | ■ Pin a specific version: <code>FROM debian:bookworm</code> , <code>FROM alpine:3.18</code> |
| ■ Password in Dockerfile or git | ■ Passwords ONLY in <code>secrets/</code> files, <code>secrets/</code> in <code>.gitignore</code> |
| ■ Port 80 accessible | ■ Do not add a server block for port 80, do not publish port 80 in compose |
| ■ <code>tail -f</code> or <code>sleep infinity</code> in entrypoint | ■ Use <code>exec</code> to run your service as PID 1 |
| ■ network: host or <code>--link</code> | ■ Use custom bridge network; containers reach each other by service name |
| ■ Bind mount instead of named volume | ■ Use named volumes with <code>driver_opts</code> to set the host path |
| ■ WordPress install wizard shows up | ■ <code>wp_setup.sh</code> must complete install before <code>php-fpm</code> starts |
| ■ Data lost after VM reboot | ■ Named volumes with <code>driver_opts</code> + <code>restart: unless-stopped</code> ensure persistence |
| ■ Missing README.md or wrong first line | ■ First line must be italicized in Markdown: <code>*This project has been created...*</code> |

11

Bonus Services

■ Bonus is only evaluated if the mandatory part is PERFECT. Complete and fully test the mandatory part before attempting bonus.

11.1 Redis Cache for WordPress

Redis is an in-memory data store used as an object cache for WordPress. It speeds up WordPress by caching database queries in RAM.

```
# srcs/requirements/bonus/redis/Dockerfile
FROM debian:bookworm
RUN apt-get update && apt-get install -y --no-install-recommends \
    redis-server \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
COPY conf/redis.conf /etc/redis/redis.conf
EXPOSE 6379
CMD ["redis-server", "/etc/redis/redis.conf"]
```

```
# srcs/requirements/bonus/redis/conf/redis.conf
bind 0.0.0.0
port 6379
maxmemory 256mb
maxmemory-policy allkeys-lru
daemonize no
```

Add to wp_setup.sh (WordPress entrypoint) to install the Redis plugin:

```
# Install Redis Object Cache plugin
wp plugin install redis-cache --activate --path=${WP_PATH} --allow-root

# Configure WordPress to use Redis
wp config set WP_REDIS_HOST redis --path=${WP_PATH} --allow-root
wp config set WP_REDIS_PORT 6379 --path=${WP_PATH} --allow-root
wp redis enable --path=${WP_PATH} --allow-root 2>/dev/null || true
```

Add to docker-compose.yml:

```
redis:
  build: ./requirements/bonus/redis
  image: redis
  container_name: redis
  restart: unless-stopped
  expose:
    - "6379"
  networks:
    - inception_net
```

11.2 FTP Server

An FTP server container pointing to the WordPress volume, allowing file uploads/downloads.

```
# srcs/requirements/bonus/ftp/Dockerfile
FROM debian:bookworm
RUN apt-get update && apt-get install -y --no-install-recommends \
    vsftpd \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
COPY conf/vsftpd.conf /etc/vsftpd.conf
COPY tools/ftp_init.sh /usr/local/bin/ftp_init.sh
RUN chmod +x /usr/local/bin/ftp_init.sh
EXPOSE 21 21000-21010
ENTRYPOINT ["/usr/local/bin/ftp_init.sh"]

# srcs/requirements/bonus/ftp/conf/vsftpd.conf
listen=YES
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_root=/var/www/html
pasv_enable=YES
pasv_min_port=21000
pasv_max_port=21010
pasv_address=127.0.0.1

#!/bin/sh
# srcs/requirements/bonus/ftp/tools/ftp_init.sh
FTP_USER=${FTP_USER:-ftpuser}
FTP_PASS=$(cat /run/secrets/db_password) # reuse or add dedicated secret

# Create FTP user if not exists
id -u $FTP_USER &>/dev/null || adduser --disabled-password --gecos '' $FTP_USER
echo "$FTP_USER:$FTP_PASS" | chpasswd

exec vsftpd /etc/vsftpd.conf
```

Add to docker-compose.yml:

```
ftp:
  build: ./requirements/bonus/ftp
  image: ftp
  container_name: ftp
  restart: unless-stopped
  env_file: .env
  secrets:
    - db_password
  volumes:
    - wordpress_data:/var/www/html
  ports:
    - "21:21"
    - "21000-21010:21000-21010"
  networks:
    - inception_net
```

11.3 Static Website (non-PHP)

A simple static site (HTML/CSS/JS or any language except PHP) served on a custom port.

```
# srcs/requirements/bonus/static/Dockerfile
FROM debian:bookworm
RUN apt-get update && apt-get install -y --no-install-recommends \
    nginx-light \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
COPY html/ /var/www/html/
COPY conf/static.conf /etc/nginx/sites-available/default
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
```

```
# srcs/requirements/bonus/static/conf/static.conf
server {
    listen 8080;
    root /var/www/html;
    index index.html;
    location / { try_files $uri $uri/ =404; }
}
```

docker-compose.yml addition:

```
static:
  build: ./requirements/bonus/static
  image: static
  container_name: static
  restart: unless-stopped
  ports:
    - "8080:8080"
  networks:
    - inception_net
```

11.4 Adminer (Database Web UI)

Adminer is a single-PHP-file database management tool accessible via browser.

```
# srcs/requirements/bonus/adminer/Dockerfile
FROM debian:bookworm
RUN apt-get update && apt-get install -y --no-install-recommends \
    php8.2-fpm php8.2-mysql wget \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# Download Adminer
RUN mkdir -p /var/www/adminer \
    && wget -O /var/www/adminer/index.php \
    https://github.com/vrana/adminer/releases/download/v4.8.1/adminer-4.8.1.php

COPY conf/adminer.conf /etc/nginx/sites-available/adminer
EXPOSE 8081
CMD ["php-fpm8.2", "-F"]
```

Alternatively, serve Adminer through NGINX by adding a location block. Access it at <https://login.42.fr/adminer/>.

11.5 Custom Service Idea

A popular choice is a **monitoring service** like Netdata or cAdvisor, or a reverse proxy dashboard like Traefik. You must be able to justify your choice and explain how it works.

```
# Example: cAdvisor (container metrics viewer)
# srcs/requirements/bonus/cadvisor/Dockerfile
FROM debian:bookworm
RUN apt-get update && apt-get install -y wget && \
    wget -O /usr/local/bin/cadvisor \
    https://github.com/google/cadvisor/releases/download/v0.49.1/cadvisor-v0.49.1-linux-amd64 && \
    chmod +x /usr/local/bin/cadvisor
EXPOSE 8082
CMD ["/usr/local/bin/cadvisor", "-port=8082"]
```

■ ■ For the custom service, be ready to explain: what the service does, why it is useful in this infrastructure, and how it integrates with the other containers.

1
2

Full File Reference

Complete list of all files that must exist in your repository. Use this as a final checklist before submission.

| File | Purpose |
|-------------|--|
| Makefile | Build orchestrator — 'make' starts everything |
| README.md | Project documentation (mandatory, specific format) |
| USER_DOC.md | End-user guide (mandatory) |
| DEV_DOC.md | Developer guide (mandatory) |
| .gitignore | Must include srcs/.env and secrets/ |

| File | Purpose |
|------------------------------|--------------------------|
| secrets/db_password.txt | MariaDB user password |
| secrets/db_root_password.txt | MariaDB root password |
| secrets/credentials.txt | WordPress admin password |

| File | Purpose |
|-------------------------|-------------------------------------|
| srcs/.env | Environment variables (git-ignored) |
| srcs/docker-compose.yml | Full stack definition |

| File | Purpose |
|-----------------|--|
| Dockerfile | FROM debian:bookworm, installs nginx + openssl |
| .dockerignore | Exclude unnecessary files from build context |
| conf/nginx.conf | HTTPS-only, FastCGI to wordpress:9000 |

| | |
|------------------------|-------------------------------------|
| tools/generate_cert.sh | OpenSSL self-signed cert generation |
|------------------------|-------------------------------------|

| File | Purpose |
|-------------------|---|
| Dockerfile | FROM debian:bookworm, installs php-fpm, wp-cli |
| .dockerignore | |
| conf/www.conf | PHP-FPM pool config, listen 0.0.0.0:9000 |
| tools/wp_setup.sh | Wait for DB, download WP, install, create users, exec php-fpm |

| File | Purpose |
|------------------|---|
| Dockerfile | FROM debian:bookworm, installs mariadb-server |
| .dockerignore | |
| conf/my.cnf | bind-address=0.0.0.0, port=3306 |
| tools/db_init.sh | Init DB on first run, exec mysqld_safe |

| File | Purpose |
|-------------------------|----------------------------------|
| redis/Dockerfile | Redis cache |
| redis/conf/redis.conf | Redis config |
| ftp/Dockerfile | vsftpd FTP server |
| ftp/conf/vsftpd.conf | FTP config |
| ftp/tools/ftp_init.sh | Create FTP user and start vsftpd |
| static/Dockerfile | Static website |
| static/conf/static.conf | NGINX serving HTML |
| static/html/index.html | Static site content |
| adminer/Dockerfile | Adminer database manager |
| cadvisor/Dockerfile | Optional custom service |

Final Tips for Success

1. Test persistence: run 'make', add a WordPress post, run 'make fclean && make', verify the post is gone (fclean wipes data). Then test that data survives 'make down && make up' (not fclean).
2. Test the reboot: actually reboot the VM, run 'make up', verify everything works. The evaluator WILL do this.
3. Test port isolation: from outside the Docker network, verify that curl http://localhost:3306 and curl http://localhost:9000 fail. Only port 443 should work.
4. Verify your admin username: log into WordPress and confirm it does not contain 'admin' or 'administrator' in any form.
5. Clean Git history: make sure you never committed .env or secrets/. Use git log --all -- srcs/.env to check.
6. Know your code: be ready to explain every line of every Dockerfile and entrypoint script. The evaluator may ask you to modify something live.
7. The 'image:' field in docker-compose.yml must match the service name — this is explicitly required by the subject.

Good luck with your Inception defense! Remember: understand every choice you made.